

Inheritance, Polymorphism, and Aggregation

A Hands-On Guide Using the Springfield Elementary Class Hierarchy

Lab Overview & Objectives

This lab exercise explores core Object-Oriented Programming (OOP) principles in C++, including public inheritance, protected member access, runtime polymorphism via virtual functions, dynamic casting, and object aggregation. You will implement a robust class hierarchy representing various individuals at Springfield Elementary School.

Class Hierarchy Architecture:

- Person (Base Class): # name (string), # age (int) | Virtual toString() | Virtual destructor
- Student (Derived from Person): # gpa (double) | Overrides toString()
- Professor (Derived from Person): # yearsExperience (int) | Overrides toString()
- Employee (Derived from Person): # ID (int) | Overrides toString()
- Department (Aggregation): Holds department name, office number, and a collection of Professor pointers.

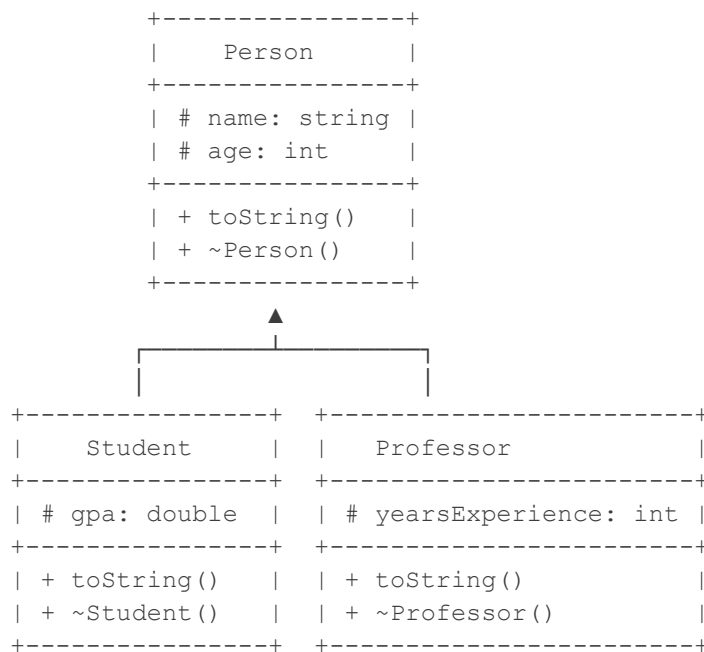


Figure 1. Partial UML diagram

Part 1 & 2: Creating the Base and Derived Classes

Implement the Person base class with protected attributes so that derived classes can access them directly. Then, implement the Student and Professor derived classes.

Part 3: Runtime Polymorphism

Declare toString() as a virtual function in the Person class. This ensures that when calling toString() through a base class pointer (Person*), the correct derived class method is invoked dynamically at runtime. Observe what happens when you make the virtual toString() method behave as pure (=0).

```
#include <iostream>
#include "Person.h"
#include "Student.h"
#include "Professor.h"
#include "Department.h"
#include "Employee.h"
using namespace std;

void experiment01() {
    cout << "experiment01 " << endl;
    Person* p1 = new Student("Bart Simpson", 10, 2.13);
    Person* p2 = new Professor("Edna Krabapple", 45, 28);

    cout << "*p1 " << p1->toString() << endl; // Calls Student's toString()
    cout << "*p2 " << p2->toString() << endl; // Calls Professor's toString()

    delete p1;
    delete p2;
}

int main() {
    experiment01();
    cout << "All done!" << endl;
}
```

Sample Output

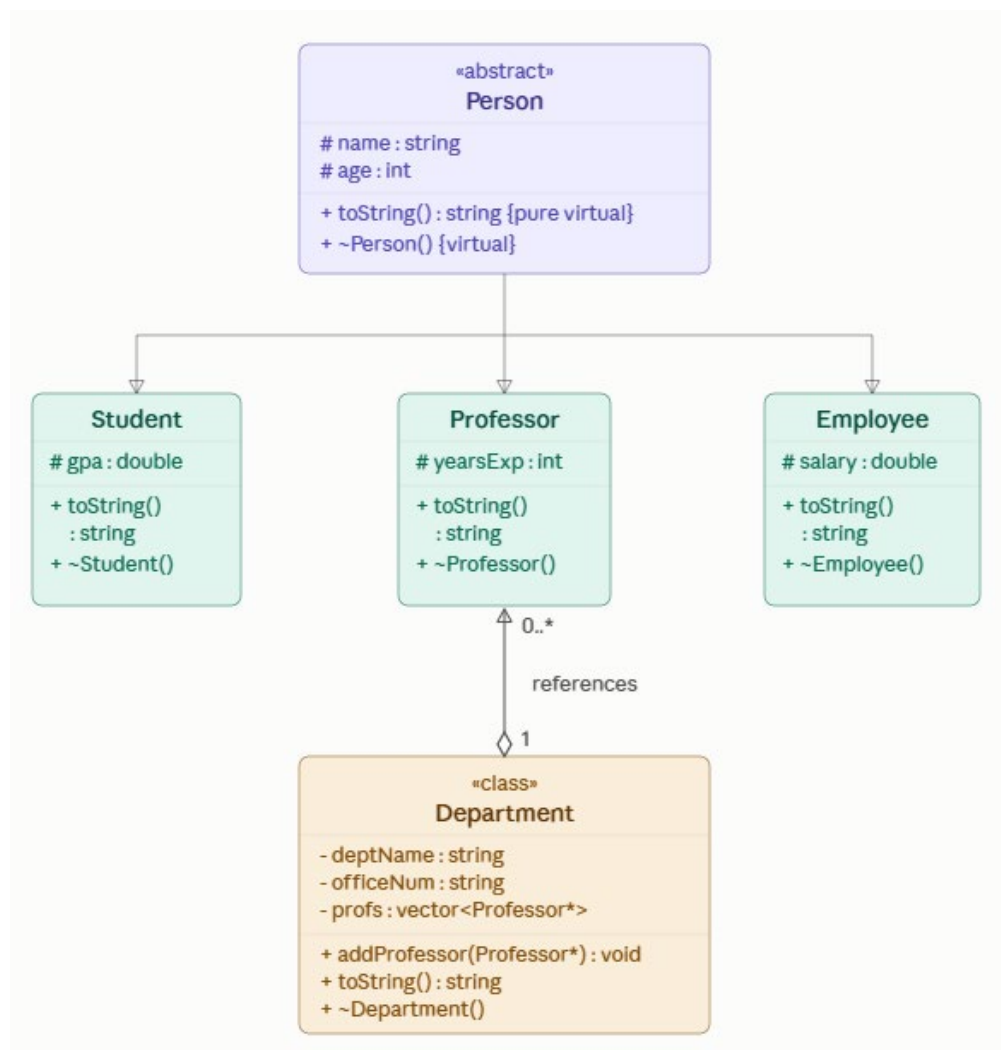
```
experiment01
*p1 000001ACD6D5C440 Student[000001ACD6D5C440 Person[Name: Bart Simpson, Age: 10], GPA : 2.13]
*p2 000001ACD6D5C4C0 Professor[ 000001ACD6D5C4C0 Person[Name: Edna Krabapple, Age: 45], Experience : 28]
+ Student destructor called for Bart Simpson
+ Person destructor called for Bart Simpson
+ Professor destructor called for Edna Krabapple
+ Person destructor called for Edna Krabapple
All done!
```

Part 4: Advanced Inheritance & Casting

1. Virtual Destructors: Ensure that destructors in Person and all derived classes are declared virtual to prevent memory leaks when deleting derived objects via base pointers.
2. Vector of Person Pointers: Store mixed objects (Student, Professor) in a `std::vector<Person*>` and iterate through them to invoke polymorphic display functions.
3. `dynamic_cast<>`: Use `dynamic_cast` to safely inspect and identify specific derived types at runtime.

Part 5: Association with Department

Create a **Department** class that encapsulates a department name, office location, and a collection (e.g., `vector<Professor*>`) representing faculty members assigned to the department. Add the **Employee** class to extend Person. It includes a salary field. Associate via aggregation (weak ownership)



Part 6 & 7: The Person-Department Model - Extended Test Suite

Below is an extended test function to validate your implementation (including Department and Employee).

```
void experiment02() {
    cout << "experiment02 " << endl;
    // Part 3: Runtime Polymorphism
    cout << "=== Part 3: Runtime Polymorphism ===\n";
    Person* p1 = new Student("Bart Simpson", 10, 2.13);
    Person* p2 = new Professor("Edna Krabapple", 45, 28);
    cout << "*p1 " << p1->toString() << endl;
    cout << "*p2 " << p2->toString() << endl;

    delete p1;
    delete p2;

    // Part 4.2: Vector of Person Pointers
    cout << "\n=== Part 4.2: Vector of Person Pointers ===\n";
    vector<Person*> people;
    people.push_back(new Student("Milhouse Van Houten", 10, 2.85));
    people.push_back(new Professor("Elizabeth Hoover", 40, 15));
    people.push_back(new Employee("Groundskeeper Willie", 50, 45000.0));
    for (const auto& person : people) {
        cout << person->toString() << endl;
    }

    // Part 4.3: Testing dynamic_cast
    cout << "\n=== Part 4.3: Testing dynamic_cast ===\n";
    for (auto person : people) {
        if (Employee* emp = dynamic_cast<Employee*>(person)) {
            cout << "Found an Employee object!\n";
            cout << emp->toString() << endl;
        }
    }

    // Part 5: Aggregation with Department
    cout << "\n=== Part 5: Aggregation Experiment ===\n";
    Professor* prof1 = new Professor("Seymour Skinner", 50, 20);
    Professor* prof2 = new Professor("Coach Krup", 42, 12);
    Department generalStudies("General Studies", "Building B, Room 101");
    generalStudies.addProfessor(prof1);
    generalStudies.addProfessor(prof2);
    cout << generalStudies.toString() << endl;;

    // Clean up dynamic memory
    for (auto person : people) {
        delete person;
    }

    delete prof1;
    delete prof2;
}
```

Sample Output

```
experiment02
=== Part 3: Runtime Polymorphism ===
*p1 000002CA2584C440 Student[000002CA2584C440 Person[Name: Bart Simpson, Age: 10], GPA : 2.13]
*p2 000002CA2584C4C0 Professor[ 000002CA2584C4C0 Person[Name: Edna Krabapple, Age: 45], Experience : 28]
+ Student destructor called for Bart Simpson
+ Person destructor called for Bart Simpson
+ Professor destructor called for Edna Krabapple
+ Person destructor called for Edna Krabapple

=== Part 4.2: Vector of Person Pointers ===
000002CA2584C440 Student[000002CA2584C440 Person[Name: Milhouse Van Houten, Age: 10], GPA : 2.85]
000002CA2584C4C0 Professor[ 000002CA2584C4C0 Person[Name: Elizabeth Hoover, Age: 40], Experience : 15]
000002CA2584C710 Employee[000002CA2584C710 Person[Name: Groundskeeper Willie, Age: 50] Salary : $45000]

=== Part 4.3: Testing dynamic_cast ===
Found an Employee object!
000002CA2584C710 Employee[000002CA2584C710 Person[Name: Groundskeeper Willie, Age: 50] Salary : $45000]

=== Part 5: Aggregation Experiment ===
000000D6B44FEAB0 --- Department [General Studies, Office: Building B, Room 101 ---
000002CA2584C790 Professor[ 000002CA2584C790 Person[Name: Seymour Skinner, Age: 50], Experience : 20]
000002CA2584C170 Professor[ 000002CA2584C170 Person[Name: Coach Krup, Age: 42], Experience : 12]

+ Student destructor called for Milhouse Van Houten
+ Person destructor called for Milhouse Van Houten
+ Professor destructor called for Elizabeth Hoover
+ Person destructor called for Elizabeth Hoover
+ Employee destructor called for Groundskeeper Willie
+ Person destructor called for Groundskeeper Willie
+ Professor destructor called for Seymour Skinner
+ Person destructor called for Seymour Skinner
+ Professor destructor called for Coach Krup
+ Person destructor called for Coach Krup
+ Department deleted 000000D6B44FEAB0 Nothing to delete

All done!
```

Faculty & Staff at Springfield Elementary



Elizabeth Hoover, B.Ed.
(Lisa's Teacher)



Edna Krabapple



Principal Skinner



Coach Krup



Lunchlady Doris



Bus driver – Otto Mann



Groundskeeper Willie